

07/01/25
Wesley
Cook

Serial Radio Basics

121

The serial radio looks almost identical to the normal serial communications.

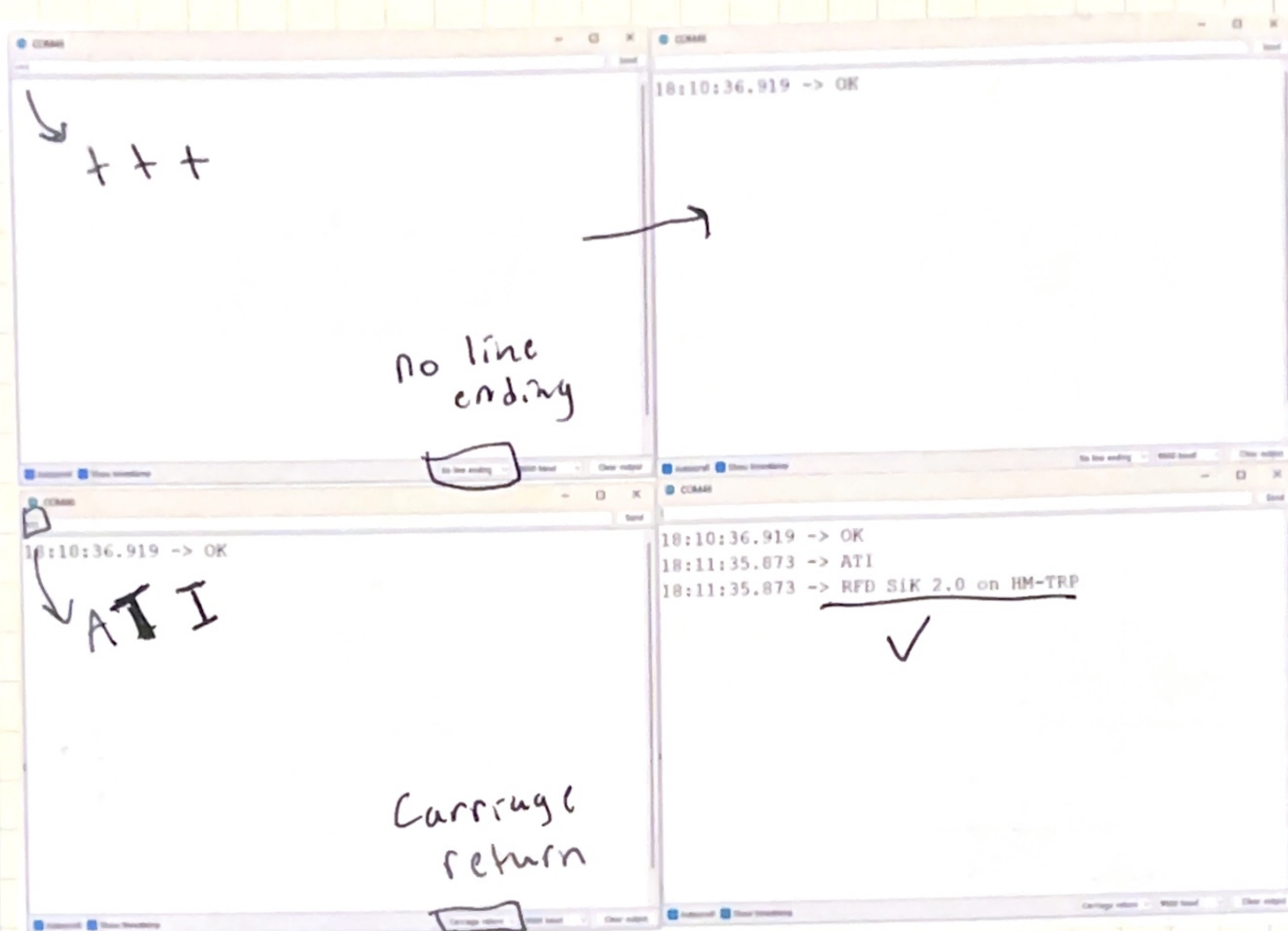
By default the radios are configured to talk to each other. To change the settings we can use the AT commands.

AT Command Code:

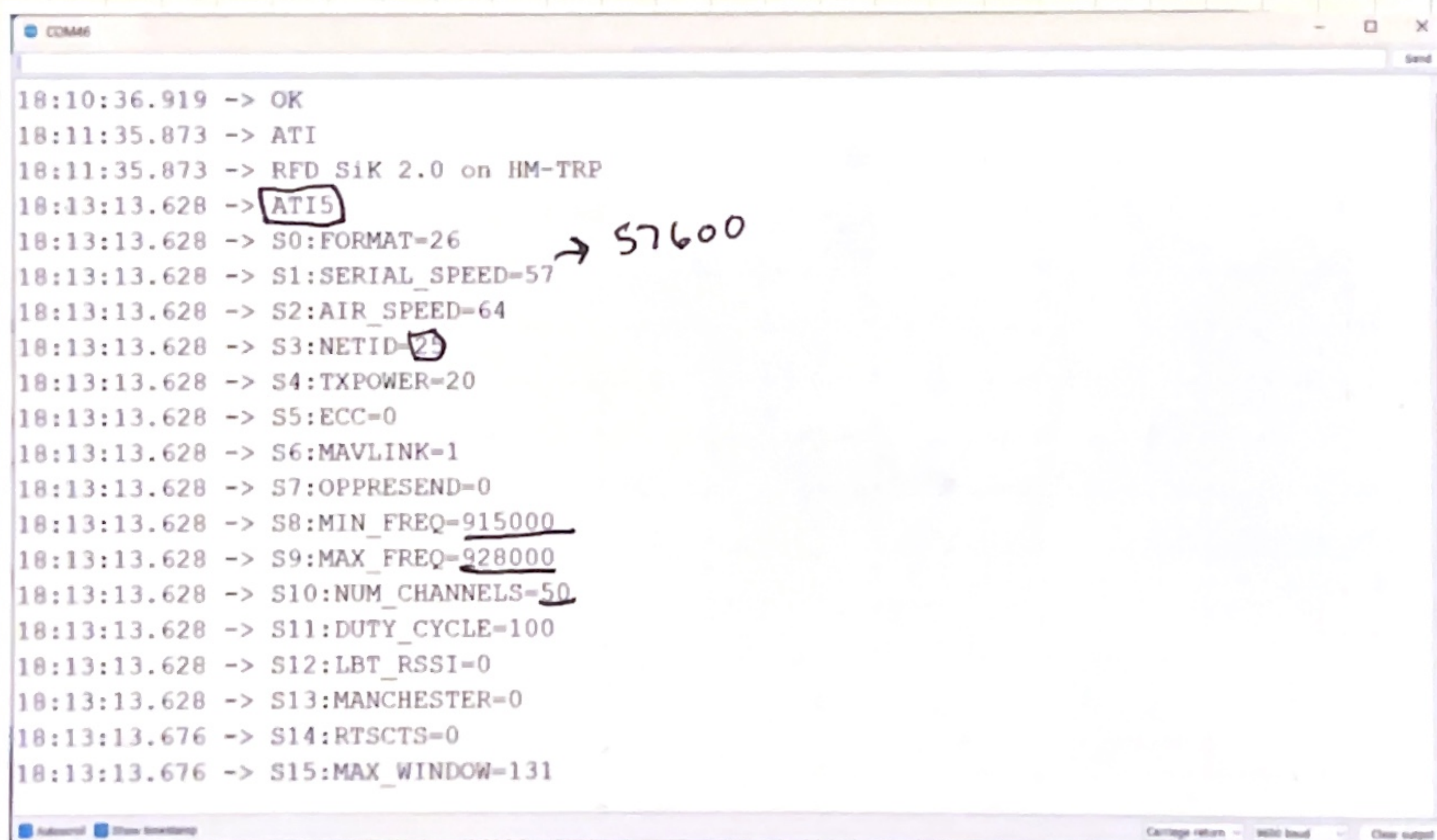
```
1 void setup()
2 {
3   Serial.begin(9600); // default
4   Serial1.begin(57600); // connected to RX (pin0), TX (pin1) on arduino giga
5 }
6
7 void loop()
8 {
9   // If we are trying to talk to the radio
10  while (Serial.available() > 0)
11  {
12    char data = Serial.read(); // Read what we want to send
13    Serial1.print(data); // Send it to the radio
14  }
15
16  // If the radio wants to talk to us
17  while (Serial1.available() > 0)
18  {
19    char data = Serial1.read(); // Read what it is saying
20    Serial.print(data); // Send it to our monitor
21  }
22 }
```

The default baud rate is 57,600.

To enter command mode we send '+++'
with no line ending. From there we can
send AT commands with the carriage return
selected.



Example of entering AT command mode
Use ATIS to see the settings:



07/01/2025
Wesley
Coker

Serial Radio Basics

123

More on AT commands:

Once in AT command mode, you can give the radio either 'AT' commands to control the local radio, or (if successfully connected) you can use 'RT' commands to control the remote radio.

The AT commands available are:

- AT1 - show radio version
- AT12 - show board type
- AT13 - show board frequency
- AT14 - show board version
- AT15 - show all user settable EEPROM parameters
- AT16 - display TDM timing report
- AT17 - display RSSI signal report
- ATO - exit AT command mode
- ATSn? - display radio parameter number 'n'
- ATSn=X - set radio parameter number 'n' to 'X'
- ATZ - reboot the radio
- AT&W - write current parameters to EEPROM
- AT&F - reset all parameters to factory default
- AT&T=RSSI - enable RSSI debug reporting
- AT&T=TDM - enable TDM debug reporting
- AT&T - disable debug reporting

all of these commands, except for ATO, may be used on a connected remote radio by replacing 'AT' with 'RT'.

Perhaps the most useful command is 'AT15' which displays all user settable EEPROM parameters. That will produce a report like this:

default
57.600

More on AT Commands

For the output of ATIS:

The first column is the S register to set if you want to change that parameter. So for example, to set the transmit power to 10dBm, use 'ATS4=10'.

Most parameters only take effect on the next reboot. So the usual pattern is to set the parameters you want, then use 'AT&W' to write the parameters to EEPROM, then reboot using 'ATZ'. The exception is the transmit power, which changes immediately (although it will revert to the old setting on reboot unless you use AT&W).

The meaning of the parameter is as follows:

- `FORMAT` - this is for EEPROM format version. Don't change it
- `SERIAL_SPEED` - this is the serial speed in 'one byte form' (see below)
- `AIR_SPEED` - this is the air data rate in 'one byte form'
- `NETID` - this is the network ID. It must be the same for both your radios
- `TXPOWER` - this is the transmit power in dBm. The maximum is 20dBm
- `ECC` - this enables/disables the golay error correcting code
- `MAVLINK` - this controls MAVLink framing and reporting. 0=no MAVLink framing, 1=frame mavlink, 2=low latency mavlink
- `MIN_FREQ` - minimum frequency in kHz
- `MAX_FREQ` - maximum frequency in kHz
- `NUM_CHANNELS` - number of frequency hopping channels
- `DUTY_CYCLE` - the percentage of time to allow transmit
- `LBT_RSSI` - Listen Before Talk threshold (see docs below)
- `MAX_WINDOW` - max transmit window in msecs, 131 is the default, 33 recommended for low latency (but lower bandwidth)

For two radios to communicate the following must be the same at both ends of the link:

- the radio firmware version
- the `AIR_SPEED`
- the `MIN_FREQ`
- the `MAX_FREQ`
- the `NUM_CHANNELS`
- the `NETID`
- the `ECC` setting
- the `LBT_RSSI` setting
- the `MAX_WINDOW` setting

the other settings may be different at either end of the link, although you will usually set them up the same at both ends.

07/01/25
Wesley
Cook

Serial Radio Basics

125

Sample Code:

```
1 float pi    = 3.14;
2 int  myInt  = 314;
3
4 void setup()
5 {
6   Serial.begin(9600);
7   Serial1.begin(57600);
8 }
9
10 void loop()
11 {
12   // Send a header byte
13   Serial1.print('R');
14   // Send our data with a return
15   Serial1.print(pi);   Serial1.print('\r');
16   Serial1.print(myInt); Serial1.print('\r');
17   Serial1.print('E'); // footer
18   delay(10000);
19 }
```

Sending
side

```
1 void setup()
2 {
3   Serial.begin(9600);
4   Serial1.begin(57600);
5 }
6
7 void loop()
8 {
9   // If the radio has something to say
10  while(Serial1.available() >= 11)
11  {
12    // Check the header
13    char header = Serial1.read();
14    if (header == 'R') // LOOK for header
15    {
16      // Parse our data
17      float pi = Serial1.parseFloat(); Serial1.read();
18      int myNum = Serial1.parseInt();  Serial1.read();
19
20      // Check that our footer is 'E'
21      if ((char) Serial1.read() == 'E')
22      {
23        Serial.print("Pi is:   "); Serial.println(pi);
24        Serial.print("myNum is: "); Serial.println(myNum);
25      }
26    }
27  }
28 }
```

Receiver
side

for
the '\r'
↓

do something
with the
data.